

MAdES — a worked example

This document is an example of a MAdES signature, and it is also the thing being signed. Everything described below happened to this file.

If you are reading it in a Markdown viewer, you are seeing the document. If you open it in a text editor, you will find a block at the bottom that the viewer hides. That block is the signature.

It replaces an earlier example, and says so

`05-a-real-signed-document.md` came first and is still in this directory. It carries a paragraph explaining that it deliberately never writes the opening marker in full, because doing so would break its own signature.

That was true when it was signed. Specification v1.4 removed the limitation, so it is no longer true — and this document proves it by writing the marker, `<!-- mades-sig`, in ordinary prose.

05 has not been edited. A signed document is not rewritten to make it say something more flattering; that is the property this whole format exists to provide, and abandoning it in the example that demonstrates it would be a strange trade. It is superseded, not corrected, and the distinction is the point.

Note also what this is *not*: the `revision / supersedes` chain of §a.7. That mechanism binds a signature to the digest of what an earlier one covered and requires the same signer and a verifying earlier block. The relationship here is editorial, and it is stated in prose that falls under this signature rather than asserted in a field that means something narrower.

What happened to this file

A signature is not applied to a document so much as *fixed around* it. Four things happen, in this order, and the order is not ours to choose.

1. **The content is frozen.** The text above and below this line is canonicalised — line endings normalised, trailing whitespace stripped — so that two systems reading the same document agree on the same bytes. From this point on, changing a single character changes what was signed.
2. **A certificate is issued.** It carries the signer’s name, their country, and a stable identifier for them. It lives for minutes, not years. That sounds fragile and is the opposite: a key that exists briefly is a key that cannot be stolen later.
3. **The signer confirms.** Not here — on the signing service’s own page, with a code sent to their own mailbox. The document is never in a position to claim that someone agreed. It can only ask, and be told.
4. **The signature is bound to these exact bytes.** The authorisation covers this document’s digest and no other. Presenting it for anything else is refused.

What you will find below the text

A block: an HTML comment opening with `<!-- mades-sig`. Every conforming Markdown renderer hides it, and every text editor shows it. Invisible in view, present in source, which is where a signature belongs.

Writing that marker in an ordinary sentence used to break a document. A verifier found blocks by scanning for the bytes and could not tell a signature from a description of one, so a paragraph like the one above produced a second, empty block — and the reader was told that a sound document could not be checked.

Since v1.4 the marker counts only at the start of a line, and never inside a fenced code block. So a document may now describe the format it is signed with:

```
<!-- mades-sig
version: 5
algorithm: ecdsa-p256
signer: someone@example.com
-->
```

Both rules are byte-level and need no Markdown parser. That is deliberate. An implementation that has to *understand* a document in order to find its signature is one that will eventually disagree with the next implementation about where the signature was — and disagreement about the location of a signature does not read to a person as a parsing difference. It reads as tampering.

Inside the block:

- **signer** and **signer-kind** — who signed, and *what* signed. A human or a machine, stated in a field that falls under the signature and mirrored by the certificate, so the two can be held against each other. Markdown is the format where people and programs take turns; this is the question it can answer that PDF never had to ask.
- **commitment** — what the signature means. Producing a document and agreeing to one are different acts, and they are recorded differently.
- **certificate-chain** — the certificate travels inside the file. Nothing has to be fetched, so nothing can fail to be fetched in five years.
- **timestamp** — an RFC 3161 token proving the signature existed while the certificate was still valid. This is what lets a short-lived key produce a long-lived signature. An expired certificate on a MAdES signature is normal, and is not a finding.
- **signature** — the bytes themselves.

There is also an **appearance** field, and it deserves a word. When this document is signed, a visible seal is added to it: a small panel with the signer's name and the moment of signing. That seal is part of what gets signed — not decoration applied afterwards. What a reader sees and what the signature covers are the same thing, which is the whole point.

How to check this yourself

Take the file. Do not trust the service that produced it, and do not trust the badge in whatever viewer you are using.

1. Strip the signature block and rebuild the signing input from what remains, following the rules in the specification.
2. Verify the signature against the certificate embedded in the file.
3. Check that certificate's chain against a trust anchor you already hold.
4. Check that the timestamp covers the signature and falls inside the certificate's validity window.

If all four hold, this document had exactly this content at the stated moment, and the named signer stood behind it. If any of them fails, you have learned something more useful than a green tick.

One thing worth adding: if your verifier reports more than one block in this file, it has not implemented §a.1 as of v1.4, and it is that verifier reporting on itself.

The PDF beside this file

The same signature ceremony produces a second artefact: this document rendered to PDF and signed as PAdES, in the same act, with the same certificate and the same confirmation. One authorisation, two representations.